

Tools: Executable Entry Points

What Is a Tool? I

A **tool** in the template repository is a directory under `tools/<scope>/<name>/` that packages one or more executable scripts behind a typed manifest (`tools.yaml`). Tools provide the third layer of the resource architecture: where funds supply static data and rules supply governance constraints, tools supply *behaviour* — computations, validation runs, and agent skill invocations that projects can trigger without re-implementing the underlying logic.

What Is a Tool? II

The tools layer deliberately mirrors the Unix philosophy of small, composable utilities that communicate through standard interfaces [Raymond2003art]. Each tool declares its entrypoints (shell scripts), its invocation contract (stdin/stdout/exit-code semantics), and its capabilities (type, version, tags) in a single manifest file. Consumers invoke tools through the `tools_invoker` module without needing to understand the tool's implementation details — a textbook application of the Facade pattern [Gamma1994design].

The tools.yaml Manifest I

Every tool root must contain a tools.yaml manifest with the following fields:

```
type: code_executor | validator | skill | agent | renderer
description: "Human-readable description of the tool"
version: "1.0.0"
tags: [curated, exemplar, production, experimental]
creator: "org/repo"
license: "Apache-2.0"
entrypoints:
  - scripts/run.sh
  - scripts/validate.sh
```

The type field determines the invocation contract the consumer should expect. The entrypoints list names the scripts that must exist on disk; the tools_invoker module validates their presence at discovery time rather than at invocation time, making failures visible early in the pipeline rather than at runtime.

The Three Template Tools I

template_code_executor

A generic code execution tool that accepts a JSON payload on standard input and returns execution results as JSON. The invocation contract is:

Entrypoint	stdin	stdout	exit code
scripts/run.sh	{"code": str, "language": str}	{"exit_code": int, "stdout": str, "stderr": str}	0 = success
scripts/validate.sh	—	Human-readable validation report	0 = valid

The code executor exemplifies tools that wrap a computational capability. The JSON-in/JSON-out contract makes it easily composable with pipeline orchestrators and agent frameworks.

The Three Template Tools II

`template_validator`

A JSON Schema validation tool. It reads a target document and a schema from disk and reports validation results in human-readable form. The entrypoint `scripts/validate.sh` exits 0 when the document is valid and non-zero with a detailed error message otherwise. The validator tool is used in the project pipeline to validate `manuscript_variables.json` against its expected schema before manuscript rendering.

`template_skill`

An agent skill invocation tool that wraps a Hermes-compatible skill definition. The entrypoint `scripts/invoke.sh` accepts a prompt string on standard input and returns the agent response as text. This tool type bridges the repository's tool architecture with external agent frameworks, demonstrating that the same manifest-and-entrypoint pattern applies equally to computational tools and AI agents.

The tools_invoker Module I

The `src/tools_invoker.py` module provides three public functions:

```
from src.tools_invoker import (
    discover_tools,
    get_tool_entrypoints,
    validate_tool_scripts_exist,
)

tools = discover_tools()
# → [{"name": "template_code_executor", "manifest": {...}}]

eps = get_tool_entrypoints("template_code_executor")
# → ["scripts/run.sh", "scripts/validate.sh"]

result = validate_tool_scripts_exist("template_code_executor")
# → {"status": "ok" | "partial" | "missing", "missing_scripts": ...}
```

The `tools_invoker` Module II

`discover_tools()` scans `tools/templates/` for subdirectories containing a parseable `tools.yaml` and returns a list of discovery records. It silently skips directories without a manifest rather than raising, ensuring that partially-completed tool directories do not block the pipeline.

`validate_tool_scripts_exist()` iterates over the manifest's `entrypoints` list and checks each path against the filesystem. It returns a structured result distinguishing between tools that are fully ready (`"ok"`), partially configured (`"partial"` — some scripts missing), and entirely absent (`"missing"`). In the current integration run, **{{integration.tools_discovered}}** tools were discovered (@fig:counts), all with valid manifests.

Tool Discovery and Reproducibility I

The tools layer contributes to reproducibility by making the *availability* of computational capabilities explicit and checkable. A project that hard-codes a path to a tool script becomes brittle when the repository is reorganised. By contrast, a project that calls `discover_tools()` and checks `validate_tool_scripts_exist()` will detect missing entrypoints at pipeline initialisation time and report them clearly, rather than failing silently at execution time [Stodden2016enhancing]. This shift from implicit to explicit dependency declaration is a key design principle of the template architecture (see `@fig:architecture`).